

Documentação de Integração - Fluxo Completo PlugQi

Visão Geral

Este documento descreve o fluxo completo de integração com o PlugQi, incluindo análise de risco via RiskSolution e posterior criação de conta escrow na QiTech.

Fluxo de Integração

1. Análise de Risco (RiskSolution)

****Pessoa Física:****

```
from risk_solution import RiskSolutionClient
import uuid
from datetime import datetime, timezone

# Inicializar cliente
rs_client = RiskSolutionClient(api_key="sua-api-key")

# Dados da pessoa física
sample_natural = {
    'id': str(uuid.uuid4()),
    'registration_id': 'reg-' + str(uuid.uuid4()),
    'registration_date': datetime.now(timezone.utc).strftime('%Y-%m-%dT%H:%M:%SZ'),
    'client_category': 'individual',
    'name': 'Fulano de Tal',
    'document_number': '123.456.789-12',
    'birthdate': '1990-05-20',
    'gender': 'male',
    'nationality': 'BRA',
    'mother_name': 'Maria de Tal',
    'father_name': 'José de Tal',
    'monthly_income': 500000, # R$5.000,00 em centavos
    'declared_assets': 2000000, # R$20.000,00 em centavos
    'occupation': 'Analista de Sistemas',
    'emails': [{'email': 'fulano.tal@example.com', 'validation_type': 'company_email'}],
    'phones': [
        {'international_dial_code': '55', 'area_code': '11', 'number': '999999999', 'type': 'mobile'}
    ],
    'address': {
        'street': 'Rua Exemplo', 'number': '123', 'neighborhood': 'Centro',
        'city': 'São Paulo', 'uf': 'SP', 'postal_code': '01001-000', 'country': 'BRA'
    },
    'source': {
        'channel': 'app', 'platform': 'ios', 'ip': '127.0.0.1',
        'session_id': str(uuid.uuid4())
    }
}

# Enviar para análise
response = rs_client.send_natural_person(sample_natural, analyze=False)
```

****Pessoa Jurídica:****

```
sample_legal = {
    'id': str(uuid.uuid4()),
    'registration_id': 'reg-' + str(uuid.uuid4()),
    'registration_date': datetime.now(timezone.utc).strftime('%Y-%m-%dT%H:%M:%SZ'),
```

```

'client_category': 'company',
'legal_name': 'Empresa Exemplo LTDA',
'trading_name': 'Empresa Exemplo',
'document_number': '08.104.627/0001-02',
'foundation_date': '2010-03-15',
'website': 'https://empresaexemplo.example',
'activity': 'Comércio varejista',
'activity_code': '47.11-3-01',
'merchant_category_code': '5411',
'tier': 'small',
'annual_revenues': 25000000, # R$250.000,00 em centavos
'emails': [{'email': 'contato@empresaexemplo.example', 'validation_type': 'company_email'}]
'phones': [
    {'international_dial_code': '55', 'area_code': '11', 'number': '33221100', 'type': 'co
},
'address': {
    'street': 'Av. Empresarial', 'number': '500', 'neighborhood': 'Bairro Industrial',
    'city': 'São Paulo', 'uf': 'SP', 'postal_code': '02000-000', 'country': 'BRA'
},
'source': {
    'channel': 'portal', 'platform': 'web', 'ip': '127.0.0.1',
    'session_id': str(uuid.uuid4())
},
'partners': [
    {
        'name': 'Sócio Um', 'document_number': '987.654.321-00',
        'birthdate': '1980-01-01', 'emails': [{'email': 'socio1@example.com'}]}
    ]
}

# Enviar para análise
response = rs_client.send_legal_person(sample_legal, analyze=False)

```

2. Verificação de Status via Webhook

```
curl --location 'https://qi.homolog.plugz.com.br/webhooks/qitech/status?id_registro=reg-12345'
```

****Resposta Esperada (Aprovado):****

```

{
  "status": "approved",
  "registration_id": "reg-12345",
  "analysis_result": {
    "risk_score": 85,
    "recommendation": "approve",
    "details": "Cliente aprovado para abertura de conta"
  },
  "timestamp": "2024-01-15T10:30:00Z"
}

```

3. Criação de Conta Escrow (Após Aprovação)

```

from plugqi import PlugQi

# Inicializar PlugQi
plugqi = PlugQi()

# Verificar conectividade
if not plugqi.health_check():
    raise Exception("Falha na conectividade com QiTech")

```

****Para Pessoa Física:****

```

# Dados da pessoa
person_data = plugqi.account_orchestrator.build_person_data(
    name="Fulano de Tal",

```

```

        document="12345678901",
        email="fulano.tal@example.com",
        birthdate="1990-05-20"
    )

    # Caminhos dos documentos
    documents_paths = {
        "rg_front": "/path/to/rg_frente.jpg",
        "rg_back": "/path/to/rg_verso.jpg",
        "proof_residence": "/path/to/comprovante.pdf"
    }

    # Executar fluxo automatizado completo
    workflow_result = plugqi.account_orchestrator.create_account_pf_complete(
        person_data=person_data,
        documents_paths=documents_paths
    )

    print(f"Status: {workflow_result['status']}")
    print(f"Account Key: {workflow_result['account_request_key']}")

```

****Para Pessoa Jurídica:****

```

# Dados da empresa
company_data = plugqi.account_orchestrator.build_company_data(
    legal_name="Empresa Exemplo LTDA",
    trading_name="Empresa Exemplo",
    document="08104627000102",
    email="contato@empresaexemplo.example"
)

# Executar fluxo PJ
workflow_result = plugqi.account_orchestrator.create_account_pj_complete(
    company_data=company_data,
    documents_paths=documents_paths
)

```

4. Operações Bancárias Pós-Abertura

```

boleto = plugqi.boleto.create_boleto(
    account_key="account-key-gerada",
    requester_profile_key="profile-key",
    boleto_data={
        "request_control_key": str(uuid.uuid4()),
        "amount": 150.0,
        "expiration": "2024-12-31",
        "payer_data": {
            "name": "Cliente Teste",
            "document_number": "12345678901"
        }
    }
)

```

```

# Enviar PIX
pix = plugqi.pix.enviar_pix_chave(
    account_key="account-key-gerada",
    pix_key="teste@email.com",
    valor=50.0
)

```

```

# Criar QR Code estático
qr_code = plugqi.pix.criar_qr_code_estatico(
    account_key="account-key-gerada",
    valor=100.0,
    descricao="Pagamento teste"
)

```

```

# TED imediata

```

```

ted = plugqi.ted.enviar_ted(
    account_key="account-key-gerada",
    target_account={
        "account_branch": "0001",
        "account_number": "123456",
        "account_digit": "7",
        "owner_document_number": "12345678901",
        "owner_name": "Beneficiário",
        "ispb": "00000000"
    },
    valor=200.0
)

# TED agendada
ted_agendada = plugqi.ted.agendar_ted(
    account_key="account-key-gerada",
    target_account=target_account,
    valor=100.0,
    data_agendamento="2024-01-16"
)

```

Implementação Recomendada

1. Estrutura do Webhook Handler

```

from flask import Flask, request, jsonify
import json

app = Flask(__name__)

@app.route('/webhooks/qitech/status', methods=['GET'])
def check_status():
    id_registro = request.args.get('id_registro')

    # Consultar status no banco de dados
    status_result = get_analysis_status(id_registro)

    if status_result['status'] == 'approved':
        # Iniciar processo de criação de conta
        account_result = create_escrow_account(status_result)
        return jsonify({
            "status": "approved",
            "account_created": True,
            "account_key": account_result.get('account_key')
        })

    return jsonify(status_result)

def create_escrow_account(analysis_result):
    """Criar conta escrow após aprovação"""
    plugqi = PlugQi()

    # Usar dados da análise para criar conta
    if analysis_result['client_category'] == 'individual':
        return create_pf_account(analysis_result)
    else:
        return create_pj_account(analysis_result)

```

2. Monitoramento e Logs

```

import logging

# Configurar logging
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

def log_integration_step(step, data, status="success"):
    """Log padronizado para steps de integração"""
    logger.info(f"INTEGRATION_STEP: {step} | STATUS: {status} | DATA: {json.dumps(data)}")

```

```
# Exemplo de uso
log_integration_step("RISK_ANALYSIS_SENT", {"registration_id": reg_id})
log_integration_step("ACCOUNT_CREATED", {"account_key": account_key})
```

Fluxo de Estados

```

graph LR
    A[Dados Cliente] --> B[RiskSolution]
    B --> C[Análise]
    C --> D[Webhook Status]
    C --> E[Aprovado]
    E --> F[PlugQi]
    F --> G[Conta Escrow]
    G --> H[Operações Bancárias]

```

Pontos de Atenção

1. ****Timeout****: Aguardar resposta da análise de risco (pode levar alguns minutos)
2. ****Retry Logic****: Implementar retry para chamadas que falharem
3. ****Validação****: Sempre validar dados antes de enviar
4. ****Logs****: Manter logs detalhados para auditoria
5. ****Segurança****: Validar assinatura dos webhooks

Teste de Integração

Execute o teste completo:

```
python test_fluxo_integrado_simples.py
```

Este teste simula todo o fluxo e gera um relatório em `relatorio\_teste\_integrado\_simples.json`.